Lab 2: Intro to Pandas

This lab will give you a chance to practice and experiment with topics from our first week of Pandas:

- Pandas data structures, especially DataFrames and Series
- Creating DataFrames
- Extracting and Maniuplating data in Series and DataFrames
- Conditional Selection in Pandas
- Helpful pandas methods for understanding data
- Sorting data by columns

Preliminaries

As a few quick reminders:

- Make sure your code exactly conforms to specifications (e.g. exactly matching variable names)
- Make sure your code works with any provided test code, and do NOT modify that test code unless explicitly stated
- When answering a specific question, ensure that all your data is within the relevant block. For text-based answers, you will be asked to edit a specific markdown cell. For code-based answers, please make sure all code is wholly contained within the relevant `BEGIN` and `END` comments.
- Please be sure to run the first cell (to import libraries) any time you work on your code. Please be sure to run all cells, from start to finish, before you submit. Make sure they run without error, that all provided tests pass, etc.

Information:

- This lab is due Friday 2/6 at 5pm. You will likely have time in class on Monday and/or Wednesday and will likely be able to work on questions 1 and 2 after Monday's lecture
- Each person should submit their own lab solo, but you are welcome to check work and bounce ideas with people near you as you go. They should not, however, give you answers and you should not divide and conquer.
- It's OK to ask AI about syntax (e.g. 'explain to me how `ones` works in NumPy), but not to answer questions for you. Since this lab is meant to teach you the syntax, try to learn it from the class notes and demos / links in the class notes, rather than just blindly googling.

In [110…

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
import random
import numpy as np
import scipy.stats as stats
```

Question 1: Practice with Pandas Series

The following code creates a Pandas series with the index specified. Imagine that this series encodes the number of squirels you counted on campus each month last Fall Semester.

1a. Use `.loc` notation to get the number of squirrels in October, storing it in `octloc`. Then use `.iloc` notation to get the number of squirrels in October, storing it in `octiloc`.

In [111…

```
# BEGIN QUESTION 1a
squirrels = pd.Series([7, 15, 25, 18, 4], index=["Aug", "Sept", "Oct", "Nov", "Dec"
octloc = squirrels.loc["Oct"]
octiloc = squirrels.iloc[2]
# END QUESTION 1a

print(squirrels)
# Sanity check tests
try:
    if octloc == octiloc:
        print("Public test for 1a passed")
    else:
        print("Public test for 1a FAILED")
except:
    print("Public test for 1a FAILED")
```

```
Aug     7
Sept    15
Oct     25
Nov     18
Dec      4
dtype: int64
Public test for 1a passed
```

1b. Now lets recreate the same dataframe without explicitly defining an index (the first line below). First, use `.loc` to get a slice of `squirrels2` from September through November (inclusive of both September and November), storing it in `sliceloc`. Then use `.iloc` to get the same slice, storing it in `sliceiloc`. Finally, use `[]` to get the same slice, storing it in `slicebracket`. It may help to run the cell and print squirrels2 before coding anything

In [112…

```
# BEGIN QUESTION 1b
squirrels2 = pd.Series([7, 15, 25, 18, 4])
sliceloc = squirrels2.loc[1:3]
sliceiloc = squirrels2.iloc[1:4]
slicebracket = squirrels2[1:4]
# END QUESTION 1b
print(squirrels2)
```

```
0     7
1    15
2    25
3    18
4     4
dtype: int64
```

1c. What is different about the indexing for slice notation using `.loc` versus `.iloc`? Which convention does `[]` follow? Your answer should only be 2-3 sentences.

Slice notation using `.loc` is right-inclusive, while it is right-exclusive for `.iloc`. `[]` follows `.iloc`'s convention.

1d. Let's go back to our original squirrels series, with our index explicitly defined. Write pandas code that uses a Boolean mask to select only counts that are at least 16, storing the result as `highcounts`.

In [113…
```
# BEGIN QUESTION 1d
squirrels = pd.Series([7, 15, 25, 18, 4], index=["Aug", "Sept", "Oct", "Nov", "Dec"
highcounts = squirrels[squirrels >= 16]
# END QUESTION 1d

print(highcounts)
```

```
Oct    25
Nov    18
dtype: int64
```

Question 2: Building Data Frames

Consider the following table.

	a	b
0	one fish	red fish
1	two fish	blue fish

2a. Create a dataframe storing the above table from list (hint: see the "From a list" example in our code demos). Save it as `dflist`.

In [114…
```
# BEGIN QUESTION 2a
dflist = pd.DataFrame([["one fish", "red fish"], ["two fish", "blue fish"]], column
# END QUESTION 2a
dflist
```

Out[114…

	a	b
0	one fish	red fish
1	two fish	blue fish

2b: Now create the same table from series, saving it as `dfseries`. Hint: you will probably want to create dictionaries as an intermediate step.

In [115…
```
# BEGIN QUESTION 2b
dict_a = pd.Series(["one fish", "two fish"])
dict_b = pd.Series(["red fish", "blue fish"])
dfseries = pd.DataFrame({"a": dict_a, "b": dict_b})
# END QUESTION 2b
dfseries
```

Out[115…

	a	b
0	one fish	red fish
1	two fish	blue fish

2c: Run the following three cells. Each extracts "blue fish" from the data frame, but they do so in three different ways. What are the three different ways "blue fish" is extracted as? How can you tell the type of data `.loc` will return just based on how you call it?

In [116…
```
dflist.loc[1, "b"]
```

Out[116…
```
'blue fish'
```

In [117…
```
dflist.loc[1:, "b":]
```

Out[117…

	b
1	blue fish

In [118…
```
dflist.loc[1:, "b"]
```

Out[118…
```
1    blue fish
Name: b, dtype: object
```

- The first one returns the data located in that entry itself, with both `.loc` parameters being both indices, pointing to row 1 column 'b'.
- The second one returns a `DataFrame` object containing that data, with both `.loc` parameters are slicing starting from row 1 and from column 'b'.
- The third one returns a `Series` object containing that data, with slicing starting from row 1, and only column 'b'.

Question 3: Practice with Small Data Frames

The dataframe `prereqs` stores information about prerequisite classes for DATA 250. Run it to take a look!

In [119…
```
prereqs = pd.DataFrame([["CSCI", 203], ["CSCI", 204], ["MATH", 216]], columns=["DEP
prereqs
```

Out[119…

	DEPT	Number
0	CSCI	203
1	CSCI	204
2	MATH	216

In this question, you'll fill in (a lot!) of slices. Specifically:

- 3a: Use `.loc` to select just `216`, storing it in q3a (it should be saved as a numpy integer, not a Series or DataFrame)
- 3b: Use `.iloc` to select just `216`, storing it in q3b (it should be saved as a numpy integer, not a Series or DataFrame)
- 3c: Use `[]` to select just `216`, storing it in q3c (it should be saved as a numpy integer, not a Series or DataFrame). Hint: remember that `[]` takes only one argument!
- 3d: Use `.loc` to select just `216`, storing it in q3d. This time, however, it should be saved as a size-1 series (not a numpy integer) where the index is 2.
- 3e: Use `.loc` to select just `216`, storing it in q3e. This time, however, it should be saved as a size-1 series (not a numpy integer) where the index is "Number".
- 3f: Use `.loc` to select just `216`, storing it in q3f. This time, however, it should be saved as a size-1 DataFrame.
- 3g: Use `.loc` to select all CSCI courses, storing it in q3g. Do NOT use a Boolean mask. Your answer should be a data frame
- 3h: Use `[]` and a Boolean mask to select all CSCI courses, storing it in q3h. Your answer should be a data frame

Some test code is provided for q2a and q2d, and you might find adding print statements helpful!

In [120…
```
# BEGIN QUESTION 3
prereqs = pd.DataFrame([["CSCI", 203], ["CSCI", 204], ["MATH", 216]], columns=["DEP
q3a = prereqs.loc[2, 'Number']
q3b = prereqs.iloc[2,1]
q3c = prereqs['Number'][2]
q3d = prereqs.loc[2:, 'Number']
q3e = prereqs.loc[2, 'Number':]
q3f = prereqs.loc[2:, 'Number':]
q3g = prereqs.loc[0:1]
q3h = prereqs[prereqs['DEPT'] == 'CSCI']
# END QUESTION 3

# Example code to help you test

try:
    print("Does q3a have value 216? ", q3a==216)
```

```
    print("Is q3a a numpy integer? ", type(q3a)==np.int64)
except:
    print("q3a not yet (correctly) defined")


try:
    print("Is q3d a series? ", type(q3d)==pd.Series)
    print("Does q3d have a single value ? ", q3d.size==1)
    print("Is that value 216? ", q3d.values[0]==216)
    print("Is the index 2? ", q3d.index[0]==2)
except:
    print("q3d not yet (correctly) defined")

# Uncomment the line below when q3h is done to verify it's the correct data frame!
q3h
```

```
Does q3a have value 216?  True
Is q3a a numpy integer?  True
Is q3d a series?  True
Does q3d have a single value ?  True
Is that value 216?  True
Is the index 2?  True
```

Out[120…

	DEPT	Number
0	CSCI	203
1	CSCI	204

Question 4: Back to Baby Names

Lets return to the baby names data set from our code demos. The code lodes this data set before. **Please change `file_name` to be some state other than PA. E.g. your home state (if not PA) or any other state that interests you.**

In [121…

```
import pandas as pd # Convention for importing pandas
import numpy as np
import os.path
import zipfile

# No need to understand this code for now - but it opens pa.txt within a .zip folde
# and loads it as a data frame!
local_filename = "namesbystate.zip"
zf = zipfile.ZipFile(local_filename, 'r')

## MODIFY THIS TO BE FOR SOME STATE OTHER THAN PA
file_name = 'NY.TXT'

field_names = ['State', 'Sex', 'Year', 'Name', 'Count']
with zf.open(file_name) as fh:
    babynames = pd.read_csv(fh, header=None, names=field_names)

babynames.sample(10)
```

Out[121…

	State	Sex	Year	Name	Count
272406	NY	M	2000	Elijah	493
84203	NY	F	1983	Maritza	44
243447	NY	M	1981	Tyquan	7
201006	NY	M	1937	Mauro	5
20171	NY	F	1938	Edith	225
234657	NY	M	1974	Ezequiel	6
266818	NY	M	1996	Kordell	5
57933	NY	F	1968	Raina	6
215367	NY	M	1957	Mike	273
293731	NY	M	2010	Paolo	7

4a. This question is a bit more open-ended. Write code to answer the following questions below, and then briefly summarize your answers in the corresponding Markdown cell. You will likely find it helpful to reference what we did in the code demos, and you might want to remember how we sorted by column in the `Working with an Extracted Column` column

- What was the most popular male name and the most popular female name in 1910?
- What was the most popular male name and the most popular female name in your birth year?
- What was the most popular male name and the most popular female name in 2024?
- In what year was your name the most popular? If your name does not show up, that's OK - just note that.
- How many distinct names (across all years) begin with the first 3 letters of your first name?
- Are there any names that include all of a Y, Z, N, and D (upper or lowercase is OK)?
- When (if ever) was the name "Ace" first used?

In [122…

```
## Code for 4a here

# What was the most popular male name and the most popular female name in 1910?
most_popular_male_name_1910 = babynames[(babynames['Year'] == 1910) & (babynames['S
print("Most popular male name in 1910: ",most_popular_male_name_1910)

most_popular_female_name_1910 = babynames[(babynames['Year'] == 1910) & (babynames[
print("Most popular female name in 1910: ",most_popular_female_name_1910)

# What was the most popular male name and the most popular female name in your birt
most_popular_male_name_2004 = babynames[(babynames['Year'] == 2004) & (babynames['S
print("Most popular male name in 2004: ",most_popular_male_name_2004)
```

```
most_popular_female_name_2004 = babynames[(babynames['Year'] == 2004) & (babynames[
print("Most popular female name in 2004: ",most_popular_female_name_2004)

# What was the most popular male name and the most popular female name in 2024?
most_popular_male_name_2024 = babynames[(babynames['Year'] == 2024) & (babynames['S
print("Most popular male name in 2024: ",most_popular_male_name_2024)

most_popular_female_name_2024 = babynames[(babynames['Year'] == 2024) & (babynames[
print("Most popular female name in 2024: ",most_popular_female_name_2024)

# In what year was your name the most popular?  If your name does not show up, that
most_popular_minh = babynames[babynames['Name'] == "Minh"].sort_values('Count', asc
print("The year when my name was most popular: ", most_popular_minh)

# How many distinct names (across all years) begin with the first 3 letters of your
number_of_names_starting_with_min = babynames[babynames['Name'].str.startswith("Min
print("Number of distinct names starting with 'Min': ", number_of_names_starting_wi

# Are there any names that include all of a Y, Z, N, and D (upper or lowercase is O
letters = 'ynzdYNZD'
names_include_those_letters = len(babynames[babynames['Name'].str.contains(letters)
print("Number of names containing letter Y, N, Z, D (both lowercase and uppercase:

first_year_of_ace = babynames[babynames['Name'] == 'Ace']['Year'].min() if len(baby
print("The year when the name Ace was first used: ",first_year_of_ace)
```

```
Most popular male name in 1910:  John
Most popular female name in 1910:  Mary
Most popular male name in 2004:  Michael
Most popular female name in 2004:  Emily
Most popular male name in 2024:  Liam
Most popular female name in 2024:  Mia
The year when my name was most popular:  2001
Number of distinct names starting with 'Min':  17
Number of names containing letter Y, N, Z, D (both lowercase and uppercase:  0
The year when the name Ace was first used:  2005
```

4a. Baby Names Analysis

Most popular names in 1910:

- Male: John
- Female: Mary

Most popular names in 2004 (birth year):

- Male: Michael
- Female: Emily

Most popular names in 2024:

- Male: Liam
- Female: Mia

Year "Minh" was most popular: The name Minh was most popular in 2001.

Distinct names beginning with "Min": There are 17 distinct names (across all years) that begin with "Min".

Names containing Y, Z, N, and D: No names in the dataset contain all four letters (Y, Z, N, and D).

First year "Ace" was used: The name Ace first appeared in 2005.

Lab Check Hint for Lab 2

For your next lab check, you should feel good about:

- Accessing data in a dataframe (using `.loc`, `.iloc`, and `[]` notation), as in questions 2-3
- Boolean masking, including using multiple conditions, as in question 4

For this lab check, I will not ask you to directly write NumPy code, but I may ask you about the output of NumPy code.

Submission Instructions

As with Lab 1, we will submit this lab as a PDF.

1. Hit "Kernel --> Restart & Run All"
2. Make sure that all of your cells run and that any output necessary for grading is shown. Be sure that you pass the provided tests.
3. Create a PDF of your notebook. To do so, (1) hit File -> Download as -> HTML (or potentially Save and File -> Export Notebook As -> HTML, depending on your version of Jupyter/OS) (2) open that HTML file in Google Chrome (I drag from downloads into a new Chrome tab), and (3) save that HTML file as a PDF (I type `ctrl+P` to print and then select `Save as PDF` as my destination). Note that this is a bit more involved than just saving as a PDF directly from Jupyter, but usually leads to better-formatted notebooks so **please follow this process exactly.**
4. Skim the PDF and make sure all your code and written responses are fully visible. If you notice anything (like your code lines are too long and get cut off; your written answers are cut off), fix those issues and repeat the above process. If I cannot see your work, it will not be graded.
5. Upload that PDF to Gradescope. If you run into any issues, please reach out to me before the deadline and document that you've completed everything (e.g. by emailing me a copy of your Jupyter notebook).

In [ ]: